Project Submission Report

Project and Repository

Project: A Secure Multi-Tenant Identity, Authorization and Enterprise Access Platform

Repository: https://github.com/krishnateja2314/iam-provider

Branch: `main`

Submitted commit: `b8a1e9cf08d513fbe2bf815914d0587a7ee6bdcb`

Commit link: https://github.com/krishnateja2314/iam-provider/commit/b8a1e9cf08d513fbe2bf815914d0587a7ee6bdcb

Submission report date: October 3, 2026

The source, configuration, tests and implementation documentation are available in the repository. A separate source archive is optional; the commit above identifies the exact submitted revision. The repository name does not imply a custom identity provider: authentication infrastructure is delegated to Keycloak.

Objective

Build a demonstrable learning-SaaS identity and access prototype in which global users can belong to multiple organizations with different roles, while protected resources remain isolated by tenant. Use established security components instead of building an identity provider, cryptography or a general policy engine.

Architecture and Selected Stack

The application is a modular monolith: one TypeScript/Fastify backend with logical authentication, user, organization, authorization, learning-resource, provisioning, privacy and audit modules. PostgreSQL stores application data. Keycloak has separate provider storage. Docker Compose starts the application and its supporting services.

Component	Selected version / responsibility
Keycloak	26.8.0; OIDC login, federation, signing keys, token issuance and provider lifecycle
PostgreSQL	16.15; relational data, constraints, transactions and quota locking
Fastify	5.12.5; TypeScript API
Casbin	5.51.1; declarative application authorization with deny precedence
JOSE	6.2.12; access-token signature and claim validation
Node.js	Container pinned to 24.21.0
Vitest / Playwright	Automated security tests and real-browser verification

Authentication and application authorization are separate. Keycloak establishes identity; the backend validates current membership, tenant context, resource ownership and required permission. Installing the provider alone does not establish tenant isolation, provisioning or privacy compliance.

Implemented Features

- Global users separate from external authentication identities and tenant memberships.
- External identities keyed by issuer plus subject, without silent email-based account merging.
- Two synthetic organizations, Acme and Globex, with different roles for the same user.
- Web and mobile-style public clients using authorization-code flow with S256 PKCE.
- Synthetic enterprise authentication through an OIDC broker realm.
- API access-token validation of signature, algorithm, issuer, audience, expiration, subject, token type and relevant claim shapes; ID tokens are rejected as API credentials.
- Uncached provider introspection for current token/session activity in the configured API.
- Central active-membership checks and tenant-filtered resource queries and lists.
- Published-resource restrictions, submission ownership and teaching-assignment-based grading.
- Casbin explicit-deny precedence, current tenant roles and subsequent policy-change decisions.
- Versioned additive migrations and database-enforced uniqueness and tenant consistency.
- Transactional organization provisioning and initial administrator bootstrap.
- Actor-scoped idempotency keys, concurrent retries, payload conflicts and rollback handling.
- Membership/resource quotas and organization suspension/reactivation.
- Purpose-specific consent, withdrawal and permitted profile-field projection.
- Queryable allow/deny/error audit records without raw credentials or tokens.
- Sensitive local mutations commit with their audit event and roll back if audit insertion fails.

All demonstration identities and organizations are synthetic. Local HTTP and public development fixtures are intended for a local academic lab, not public deployment.

Verification and Actual Evidence

The complete recorded run on **October 2, 2026, at 15:56:38 UTC** passed:

Verification	Recorded result
Complete automated suite	159 passed, 0 failed, 0 skipped across 7 suites
Authorization suite	59 cases, including 48 role/resource/action combinations and 11 boundaries/change cases
Default suite without live-provider gate	153 passed, 6 intentionally skipped live cases
Type checking and production build	Passed
npm audit at verification time	0 reported vulnerabilities

Verification	Recorded result
Real-browser PKCE and enterprise paths	Passed for web and mobile-style clients
Tenant and session browser checks	Allowed/denied boundaries, replay and ownership/revocation passed
Separate new-volume startup	Services initialized and browser proof passed while original databases were preserved

Tests use real PostgreSQL. Token fixtures exercise the production JOSE validator with signed tokens; separate live cases use actual Keycloak-issued tokens and the running API. Browser verification is additional evidence, not part of the 159-test Vitest count. Skipped tests are not counted as passing tests.

These are dated observations, not a new October 3 test run or a production security certification. First-start verification used separate fresh volumes and the build context; an independent Git-clone-based execution was not performed in that run.

Session Lifecycle Findings

Keycloak owns token issuance, refresh rotation and provider session lifecycle. The backend's online introspection provides the configured enforcement mechanism for already-issued access tokens on subsequent requests.

- Logout, own-session revocation and logout-all denied subsequent existing-token API requests.
- Refresh replay rejected reused and successor refresh tokens, and denied successor access before explicit logout.
- Password reset rejected the old password and accepted the new password, but left the existing access session valid.
- Incident response must also use logout-all when existing sessions must be invalidated.
- Final local operation-to-first-denial measurements were 62, 65 and 38 milliseconds across three samples; these are not an SLA or a guarantee that in-flight operations are cancelled.

Provider session references do not prove hardware-device identity or attestation.

Reproduce and Demonstrate

Clone the repository, select the submitted revision, and start the services:

```
git clone https://github.com/krishnateja2314/iam-provider.git
cd iam-provider
git checkout b8a1e9cf08d513fbe2bf815914d0587a7ee6bdcb
docker compose up -d --build --wait
```

Prerequisite: Docker Desktop with Linux containers. Host Node24+ is additionally needed for configuration and browser-verification scripts.

Demonstration	Local address
Web client	http://localhost:5173/web.html
Mobile-style client	http://localhost:5173/mobile.html
API	http://localhost:3100
Keycloak	http://localhost:8180

Sign in as synthetic `alice` / `alice123`. Select Acme to demonstrate administrator access, then Globex to demonstrate learner access and administrator denial. Use the demo guide for instructor, federation, audit and session scenarios.

Default security tests:

```
docker compose exec -T backend env KEYCLOAK_INTROSPECTION=false RUN_DATABASE_TESTS=1 npm test
```

Expected fixture result: 153 passed and 6 intentionally skipped live cases. The demo guide provides cleanup-safe instructions for enabling the test-only client and running all 159 tests. The child-process setting does not disable introspection in the running API server. Do not delete volumes to update existing realm configuration.

Documentation for the Mentor

All links below are pinned to the submitted revision:

- Whole repository report
- Step-by-step demo and explanations
- Actual execution evidence
- Requirements-to-test traceability
- Domain model
- Threat model
- Architecture decisions

Limitations and Next Work

Application machine principals, user delegation, full SCIM, native mobile clients, hardware-device attestation and the optional controlled performance experiment are deferred. Enterprise federation is demonstrated using synthetic realms rather than a real company domain/IdP. The provider-admin infrastructure service account is not an application agent feature.

Further production work would require deployment hardening, TLS and secret management, independent review, and broader operational/security validation. A zero-advisory npm audit and a passing test suite do not prove absence of vulnerabilities.

AI Assistance and Student Responsibility

Material AI assistance was used for implementation and documentation and is disclosed in the repository report. Both students should be able to explain the actual request flow, evaluate a selected function/test and make a small change while preserving the security invariants. Execution evidence is not a substitute for that understanding.

Submission

This report can be uploaded to the mentor's Drive folder together with the demo guide and verification evidence. Source is available through the repository and pinned commit; the source ZIP is only an optional backup unless specifically required. Ensure the mentor has access if the repository is private.